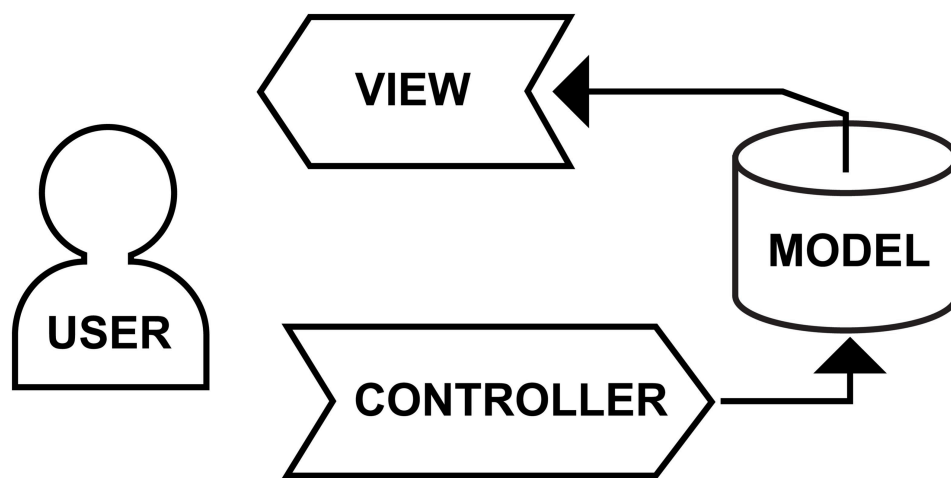


productController.js

File `controllers\productController.js` của bạn hoàn toàn đúng cú pháp và logic cơ bản cho một Controller (Bộ điều khiển) xử lý các chức năng CRUD (Create, Read, Update, Delete) sản phẩm.

Tôi sẽ giải thích chi tiết về cấu trúc, cú pháp và cách vận hành của các hàm trong Controller này, dựa trên mô hình **MVC (Model-View-Controller)**.



MODEL - VIEW - CONTROLLER PATTERN

Shutterstock

Giải thích Cấu trúc Controller

Controller (`productController.js`) là nơi chứa các hàm xử lý logic kinh doanh (business logic) của ứng dụng. Mỗi hàm này (gọi là **Action** hoặc **Route Handler**) sẽ nhận Request (yêu cầu) từ client, tương tác với Model (Cơ sở dữ liệu), và cuối cùng gửi Response (phản hồi) trở lại client, thường là thông qua View (Giao diện).

1. Khai báo Modules

```
const Product = require("../models/Product")
const fs=require("fs")
```

- `const Product = require("../models/Product")` : Tải **Model Product** (Schema Mongoose) từ thư mục `models` . Model này là giao diện để bạn tương tác với bộ sưu tập (Collection) sản phẩm trong MongoDB.
- `const fs = require("fs")` : Tải module **fs (File System)** tích hợp sẵn của Node.js. Module này dùng để làm việc với hệ thống tệp tin (ví dụ: đọc, ghi, xóa file). Mặc dù bạn chưa dùng nó trong các hàm hiện tại, việc import cho thấy bạn đã chuẩn bị cho các chức năng liên quan đến việc xử lý tệp tin (ví dụ: xóa file ảnh cũ khi cập nhật hoặc xóa sản phẩm).

2. Hàm `getAllProduct` (Read - Đọc danh sách)

```
const getAllProduct = async (req, res) => {
  const products = await Product.find();
  const {msg}=req.query;

  res.render("list", { products,msg });
}
```

- **Chức năng:** Lấy tất cả sản phẩm từ cơ sở dữ liệu và hiển thị chúng.
- `Product.find()` : Phương thức của Mongoose để **truy vấn tất cả các tài liệu** (documents) trong Collection `products` .
 - Sử dụng `await` vì thao tác truy vấn CSDL là **bất đồng bộ (asynchronous)**.
- `const {msg} = req.query;` : Lấy giá trị của tham số `msg` từ **chuỗi truy vấn (query string)** trên URL (ví dụ: `/products?msg=add-success`). Tham số này thường dùng để hiển thị thông báo thành công hoặc thất bại sau khi thực hiện các hành động khác (như thêm, xóa).
- `res.render("list", { products, msg });` : Gửi phản hồi bằng cách **kết xuất (render) file View** có tên là `list.ejs` (vì bạn đã cấu hình View Engine là EJS). Dữ liệu `products` và `msg` được truyền sang file View để hiển thị.

3. Hàm `getFormProduct` (Create - Hiển thị form thêm)

```
const getFormProduct = async (req, res) => {
  res.render("add")
}
```

- **Chức năng:** Hiển thị trang giao diện để người dùng nhập thông tin sản phẩm mới.

- `res.render("add")` : Kết xuất file View có tên là `add.ejs` . Đây là hàm xử lý cho route GET (`/products/add`).

4. Hàm `createProduct` (Create - Xử lý thêm)

```
const createProduct=async(req,res)⇒{
  const data=req.body;
  if(req.file){
    data.image=req.file.filename;
  }
  await Product.create(data)
  res.redirect("/products?msg=add-success")
}
```

- **Chức năng:** Xử lý dữ liệu form gửi lên (thường là POST) và lưu sản phẩm mới vào CSDL.
- `const data = req.body;` : Lấy **dữ liệu trường** (ví dụ: tên, giá) được gửi từ form. Dữ liệu này có được nhờ middleware `express.urlencoded()` đã cài đặt trong `app.js` .
- `if (req.file) { data.image = req.file.filename; }` : Logic này cho thấy bạn đang sử dụng middleware `multer` (đã thấy trong `package.json`) trên Route. Nếu có tệp tin được tải lên, thông tin của nó (đặc biệt là tên tệp đã lưu trên server: `req.file.filename`) sẽ được gán vào trường `image` của đối tượng `data` .
- `await Product.create(data)` : Phương thức của Mongoose để **tạo và lưu** một tài liệu mới vào CSDL bằng dữ liệu đã thu thập.
- `res.redirect("/products?msg=add-success")` : Chuyển hướng (Redirect) người dùng trở lại trang danh sách sản phẩm, kèm theo thông báo thành công qua query string.

5. Hàm `deleteProduct` (Delete - Xóa)

```
const deleteProduct=async(req,res)⇒{
  const {_id}=req.params;
  await Product.findByIdAndDelete(_id)
  .then(result⇒{
    res.redirect("/products?msg=delete-success")
  })
  .catch(err⇒{
    res.redirect("/products?msg=delete-failed")
  })
}
```

- **Chức năng:** Xóa một sản phẩm dựa trên ID.

- `const {_id} = req.params;` : Lấy giá trị ID (`_id`) từ **tham số URL** (URL parameters). Ví dụ: nếu URL là `/products/delete/12345` , thì `_id` là `12345` . (Điều này đòi hỏi route phải được định nghĩa là `/products/delete/:_id`).
 - `Product.findByIdAndDelete(_id)` : Phương thức của Mongoose để **truy vấn tài liệu theo ID và xóa nó**.
 - `.then()` / `.catch()` : Xử lý kết quả của thao tác xóa (là một Promise):
 - **Thành công (`.then`)**: Chuyển hướng về trang danh sách với thông báo thành công.
 - **Thất bại (`.catch`)**: Chuyển hướng về trang danh sách với thông báo thất bại (có thể do ID không tồn tại hoặc lỗi kết nối CSDL).
-

6. Xuất Module

```
module.exports = {
  getAllProduct,
  getFormProduct,
  createProduct,
  deleteProduct
}
```

- `module.exports = { ... }` : Xuất tất cả các hàm đã định nghĩa để chúng có thể được `require()` và sử dụng trong file Router chính (ví dụ: `rootRouter.js`).